



UNIVERSITY OF SCIENCE - VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW5 - AUTOMATION TESTING

Software Testing Project Report

Authors:

Lưu Thanh Thuý

(22127410)

ltthuy22@clc.fitus.edu.vn

Supervisors:

Teacher Trần Duy Hoàng

Teacher Hồ Tuấn Thanh

Teacher Trương Phước Lộc

August 1, 2025

Table of Contents

1	Group Information	1
2	Automation Process Step by Step	1
2.1	Test Data Preparation	1
2.2	Script Initialization	1
2.3	Test Execution	1
2.4	Result Logging and Cleanup	2
3	Implementation of Data-Driven Techniques	2
4	Use of Checkpoints/Assertions in Scripts	2
5	Results of the Automated Tests	2
6	Video demonstration	3
6.1	Feature 1: Sign In	3
6.2	Feature 2: User Management (Admin Only)	3
7	Self-Assessment	3

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyên Nhi	22127310	- SignUp	Done
		- Checkout	Done
Lưu Thanh Thuý	22127410	- SignIn	Done
		- User Management	Done
Nguyễn Phước Minh Trí	22127424	- Catalog	Done
		- Categories	Done
Võ Lê Việt Tú	22127435	- MyProfile	Done
		- Order Management	Done
Trần Thị Cát Tường	22127444	- Contact	Done
		- Category Management	Done

2 Automation Process Step by Step

2.1 Test Data Preparation

- **User Management:**
 - Test cases for add, edit, delete user are stored in `user_manage_data.csv`.
 - Each row includes all required fields (First Name, Last Name, DOB, Address, Postal code, City, State, Phone, Email, Password, Action, Expected Result).
 - Edge cases (empty, invalid, duplicate, future date, etc.) are covered.
- **Sign In:**
 - Test cases for login are stored in `login_test_data.csv`.
 - Each row includes email, password, expected result, and actual result for validation.
 - Covers valid, invalid, empty, injection, and boundary cases.

2.2 Script Initialization

- Python scripts use Selenium WebDriver for browser automation.
- Test data is loaded using pandas, mapped to form fields and actions.
- Browser drivers (Chrome, Firefox, Safari) are initialized for cross-browser testing.
- Logging and output folders are created for each run.

2.3 Test Execution

- **User Management:**
 - For each test case, script navigates to user management page.
 - Performs action (add, edit, delete) based on `Action` field.
 - Fills all form fields dynamically, handles dropdowns, date pickers, overlays.

- Submits form, waits for UI feedback, and handles exceptions.
- **Sign In:**
 - For each test case, script navigates to login page.
 - Inputs email and password from CSV.
 - Submits login form, waits for dashboard or error message.
 - Handles edge cases (invalid format, empty, injection, long/short values).

2.4 Result Logging and Cleanup

- After each test, actual results (UI messages, redirects) are captured.
- Results are logged to output files per browser and feature.
- Screenshots can be captured for failed cases (optional).
- Browsers are closed and resources released after all tests.

3 Implementation of Data-Driven Techniques

- Test scenarios and input values are externalized in CSV files for both features.
- Script iterates through each row, mapping columns to form fields and actions.
- Handles missing/invalid fields by checking for NaN or empty values.
- Data-driven design enables easy scaling, maintenance, and regression testing.
- Example mapping:
 - `row['First Name']` → input field with id `firstname`
 - `row['Postal code']` → input field with id `postcode`
 - `row['Email'], row['Password']` → login form fields
 - `row['Action']` → determines add, edit, delete user

4 Use of Checkpoints/Assertions in Scripts

- **User Management:**
 - After submitting form, script checks for success/error messages (e.g., `alert-success`, `alert-danger`).
 - Compares actual message to expected result from CSV.
 - Checks field-level errors (e.g., “First Name is required”, red highlight).
 - Verifies user list is updated after add/edit/delete.
- **Sign In:**
 - After login, script checks for dashboard loaded or error message.
 - Compares actual result to expected result from CSV.
 - Checks for specific error messages (invalid email, password required, locked user).
 - Verifies page redirect or error display.
- All assertions are logged with PASS/FAIL, test case ID, and details.
- Failed assertions include error messages and optionally screenshots.

5 Results of the Automated Tests

Results are recorded for each test case and browser

- Status (PASS/FAIL) is logged for each scenario, with expected and actual outcomes.
- Output files (e.g., logs/uc02_results_chrome.txt, logs/uc01_results_chrome.txt) contain:
 - Test case ID
 - Expected result
 - Actual result
 - Status
 - Timestamp (optional)

Summary

- Automation detected both successful and failed scenarios for both features, verifying system behavior and identifying issues for developers.
- Results can be exported or visualized for test coverage and quality metrics.

6 Video demonstration

6.1 Feature 1: Sign In

[Automation Process for SignIn feature](#)

6.2 Feature 2: User Management (Admin Only)

[Automation Process for User Management \(Admin Only\)](#)

7 Self-Assessment

No.	Criteria	Grade	Self-Assessed
1	Sign In	50	50
1.1	Report + Bug	15	15
1.2	Test cases	5	5
1.3	Script files	10	10
1.4	Data files	10	10
1.5	Videos	10	10
2	User Management	50	50
2.1	Report + Bug	15	15
2.2	Test cases	5	5
2.3	Script files	10	10
2.4	Data files	10	10
2.5	Videos	10	10
	Total	100	100